

Cascade

RECURSIVE MONEY FLOW GRAPHS ON FLOWVAULT

One deposit. A directed acyclic graph of FlowVault vaults executes. The keeper automaton chains Lock, Split, and Hold at every downstream node until the entire cascade settles.

Built on Stacks

FlowVault SDK

Stacks Testnet

3

Primitives

2

Templates

1

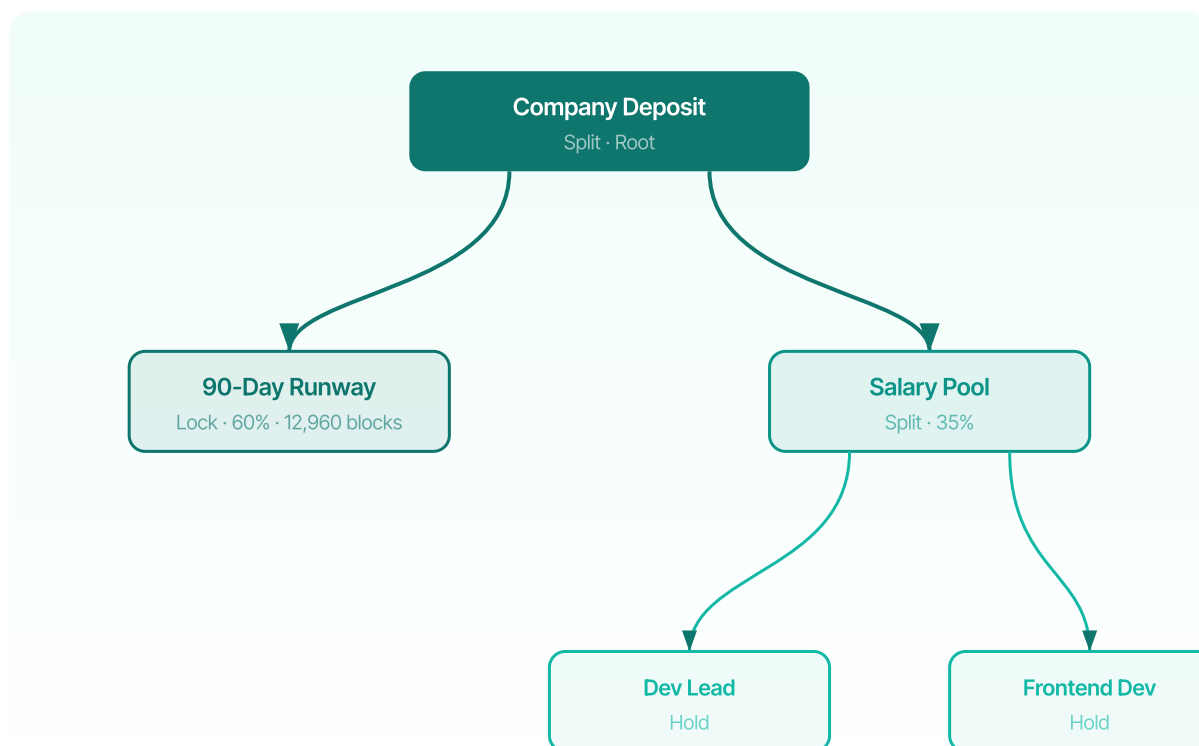
Contract

100%

On-Chain

What Cascade Does

Most financial applications only *move* money. Cascade defines how money *behaves* after deposit. Builders define a directed acyclic graph where each node is a FlowVault routing rule. The keeper automaton walks the graph in topological order — configuring routing rules, executing deposits, and cascading funds to downstream nodes — until every leaf node receives its allocation.



FlowVault Integration

Cascade uses all three FlowVault primitives in combination, orchestrated by a DAG engine. The keeper walks nodes in topological order, calling the FlowVault SDK at each step.



Lock

Time-bound escrow at each node. Funds are locked until a specific Stacks block height. Used for runway reserves, milestone locks, and vesting schedules.



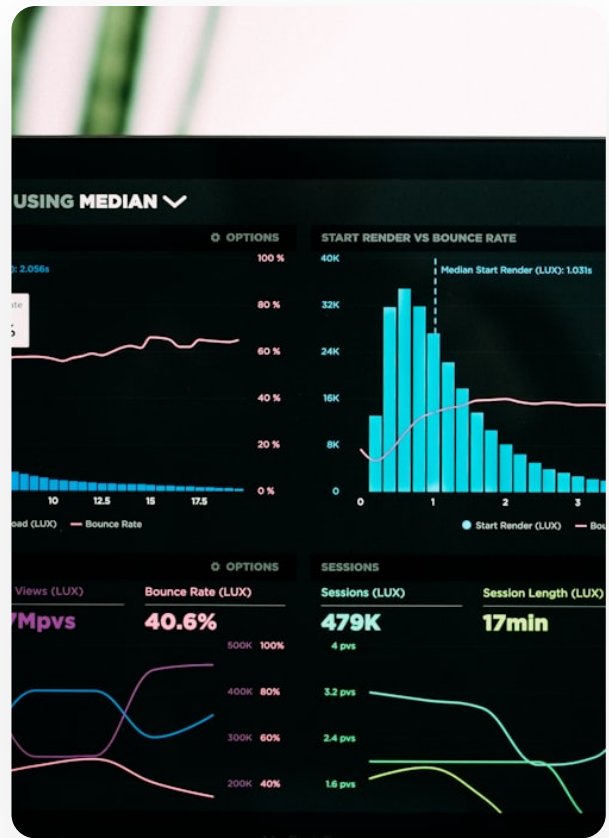
Split

Instant routing to recipient addresses. A percentage of incoming funds is sent to a designated Stacks principal — salaries, DAO contributors, agent commissions.



Hold

Liquid pass-through. The remaining balance after Lock and Split stays available and cascades to downstream child nodes, powering the recursive flow.



How It Works

A directed acyclic graph of FlowVault vaults. One deposit. Automated execution. On-chain settlement.

1 ROOT DEPOSIT	3 PRIMITIVES	∞ GRAPH DEPTH	100% ON-CHAIN
-----------------------	---------------------	----------------------	----------------------

EXECUTION FLOW

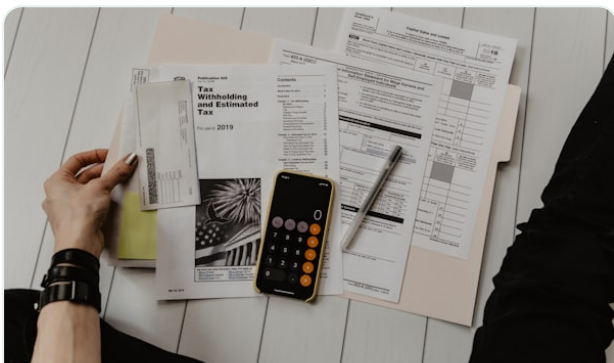
STEP	ACTION	FLOWVAULT CALL
1	Validate DAG (no cycles, single root)	—
2	Topological sort nodes	—
3	For each node: clear previous routing	<code>clearRoutingRules()</code>
4	Set Lock + Split rules for node	<code>setRoutingRules({lockAmount, splitAddress, splitAmount})</code>
5	Deposit funds into node's vault	<code>deposit(amount)</code>
6	Calculate downstream allocation	—
7	Repeat for all children in order	Loop to step 3

```
// Cascade keeper automaton – processes nodes in topological order for
(const nodeId of topologicalSort(graph)) { const alloc =
calculateNodeAllocation(node, totalMicro, graph); await
sdk.clearRoutingRules(); const tx = await sdk.setRoutingRules({
lockAmount: alloc.lockAmount, lockUntilBlock: lockUntil, splitAddress:
```

```
alloc.splitAddress, splitAmount: alloc.splitAmount, }); await  
sdk.deposit(totalMicro); // Cascade remainder to children }
```

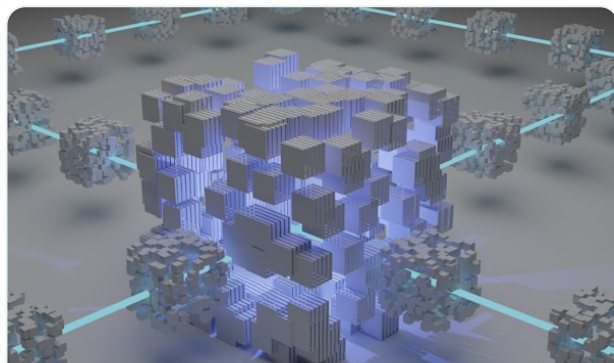
What You Can Cascade

Every multi-step money flow becomes a single deposit. No manual transfers. No spreadsheets.



Payroll Cascade

One deposit → salaries distributed, runway locked, emergency reserves filled. Entire org paid in one transaction.



DAO Treasury

Community funds auto-route to working groups, core reserves, and contributor pools. Governance enforced on-chain.



Revenue Splits

Product revenue split across team, investors, and ops — recursively. No manual transfers.



Milestone Escrow

Client funds locked. Milestone met → portion released. Sub-milestones cascade to subcontractors.

Smart Contract

`cascade-registry-v2` stores published cascade graph definitions on-chain. Graphs are discoverable, forkable, and verifiable.

```
;; Cascade Registry – On-Chain Graph Storage ;; Stores graph metadata
(name, description, node/edge counts, creator) (define-read-only (get-
graph (graph-id uint)) (match (map-get? graphs { graph-id: graph-id })
graph (ok graph) ERR-GRAPH-NOT-FOUND)) (define-read-only (get-graphs-by-
creator (creator principal)) (ok (default-to u0 (get count (map-get?
creator-graph-count { creator: creator })))))) (define-public (register-
graph (name (string-ascii 64)) (description (string-ascii 256)) (node-
count uint) (edge-count uint)) ;; Validates name length, stores graph
metadata, increments counter ...)
```

Deployed on Stacks Testnet:

ST3EH039PNE6HSXYV1CY0AC58SA0N06GMHS3CQB9H.cascade-registry-v2

Architecture

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend	Next.js 16, React 19, TypeScript	Landing page, graph editor, wallet integration
Graph Engine	TypeScript	DAG validation, cycle detection, topological sort, allocation math
Cascade Keeper	TypeScript + FlowVault SDK	Multi-node FlowVault orchestrator, automated cascade execution
Smart Contract	Clarity	On-chain graph registry — publish, discover, fork cascade templates
FlowVault Layer	Clarity (Stacks testnet)	Lock, Split, Hold primitives for programmable USDCx routing
Wallet	@stacks/connect	Leather, Xverse wallet integration for transaction signing
Testing	Vitest, Clarinet	16 unit tests for graph engine + cascade flow logic

Deployments & Links

[Live Demo](#)[Editor](#)[GitHub Repository](#)[Contract on Explorer](#)

Cascade · Recursive Money Flow Graphs · Built with FlowVault on Stacks